

Computer Architecture Lab

(EL-4031)

Project Design
Spring Semester 2026

Simple Microprocessor Design



Name

Muhammad Ibrahim

Roll No

23i – 6090

Section

B

Date of Submission

28/04/2026

Department of Electrical Engineering
National University of Computer and Emerging Sciences, Islamabad

Description:

Designed and built a simple microprocessor from scratch using both Logisim for the hardware simulation and Verilog for the coding part. The project includes all the essential components like a Control Unit, an ALU for math operations, and a Register File to store data. A 16-bit instruction format is set that allows the processor to fetch instructions from memory, decode them, and perform tasks like addition, subtraction, and data storage. It was a great way to learn how hardware and software actually interact, and I managed to get the full cycle working so that it can run basic programs automatically.

Verilog Code for each Module:

1) Program Counter (PC 4-bit):

```
module PC_4bit(
output reg [3:0] pc,
input [3:0] pc_next,
input clk,
input clr
);

always @(posedge clk or posedge clr)

if(clr)
pc <= 0;
else
pc <= pc_next;

endmodule
```

2) Instruction Memory (IM $2^4 \times 16$):

```
module IM_16x16(
input [3:0] addr,
output reg [15:0] instruction
);

reg [15:0] mem [0:15];

initial begin
mem[0] = 16'h0645;
mem[1] = 16'h0802;
mem[2] = 16'h036C;
mem[3] = 16'h0890;
mem[4] = 16'h0F52;
mem[5] = 16'h0000;
mem[6] = 16'h0000;
mem[7] = 16'h0000;
mem[8] = 16'h0000;
```

```
mem[9] = 16'h0000;
mem[10] = 16'h0000;
mem[11] = 16'h0000;
mem[12] = 16'h0000;
mem[13] = 16'h0000;
mem[14] = 16'h0000;
mem[15] = 16'h0000;
end

always @(*) instruction = mem[addr];

endmodule
```

3) Register File (RF 2⁴x4):

```
module dec_2to4(
input [1:0] sel,
input en,
output reg [3:0] out
);

always @(*) begin

if(en)

case(sel)
2'b00: out = 4'b0001;
2'b01: out = 4'b0010;
2'b10: out = 4'b0100;
2'b11: out = 4'b1000;
default: out = 4'b0000;
endcase

else out = 4'b0000;

end

endmodule

module reg_4bit(
input [3:0] d,
input clk,
input clr,
input load,
output reg [3:0] q
);
```

```
always @(posedge clk or posedge clr)
```

```
if(clr)
```

```
q <= 0;
```

```
else if(load)
```

```
q <= d;
```

```
endmodule
```

```
module mux_4to1(
```

```
input [3:0] in0, in1, in2, in3,
```

```
input [1:0] sel,
```

```
output reg [3:0] out
```

```
);
```

```
always @(*) begin
```

```
case(sel)
```

```
2'b00: out = in0;
```

```
2'b01: out = in1;
```

```
2'b10: out = in2;
```

```
2'b11: out = in3;
```

```
default: out = 4'b0000;
```

```
endcase
```

```
end
```

```
endmodule
```

```
module RF_4x4(
```

```
input clk,
```

```
input clr,
```

```
input RegWR,
```

```
input [1:0] raddr1,
```

```
input [1:0] raddr2,
```

```
input [1:0] waddr,
```

```
input [3:0] wdata,
```

```
output [3:0] rdata1,
```

```
output [3:0] rdata2
```

```
);
```

```
wire [3:0] load;
```

```
wire [3:0] q0, q1, q2, q3;
```

```
dec_2to4 d(waddr, RegWR, load);
```

```
reg_4bit r0(wdata, clk, clr, load[0], q0);
reg_4bit r1(wdata, clk, clr, load[1], q1);
reg_4bit r2(wdata, clk, clr, load[2], q2);
reg_4bit r3(wdata, clk, clr, load[3], q3);

mux_4to1 m1(q0, q1, q2, q3, raddr1, rdata1);
mux_4to1 m2(q0, q1, q2, q3, raddr2, rdata2);

endmodule
```

4) Arithmetic Logic Unit (ALU 4-bit):

```
module ALU_4bit(
input [3:0] A,
input [3:0] B,
input [2:0] op,
output reg [3:0] result,
output reg zf
);

always @(*) begin

case(op)
3'b000: result = A & B;
3'b001: result = A | B;
3'b010: result = A + B;
3'b011: result = A - B;
3'b111: result = (A < B) ? 4'b0001 : 4'b0000;
default: result = 4'b0000;
endcase

zf = (result == 0);

end

endmodule
```

5) Data Memory (DM 2⁴x4):

```
module DM_16x4(
input clk,
input clr,
input MRD,
input MWR,
input [3:0] addr,
input [3:0] wdata,
output reg [3:0] rdata
);
```

```
reg [3:0] mem[0:15];

integer i;

always @(posedge clk or posedge clr) begin

if(clr)
for(i = 0; i < 16; i = i + 1)
mem[i] <= 0;
else if(MWR)
mem[addr] <= wdata;

end

always @(*) begin

if(MRD)
rdata = mem[addr];
else
rdata = 4'b0000;

end

endmodule
```

6) Control Unit (CU):

```
module CU(
input [3:0] opcode,
output reg RegWR,
output reg MRD,
output reg MWR,
output reg MemtoReg,
output reg ALUSrc,
output reg Branch,
output reg Jump,
output reg RegDst,
output reg BranchType,
output reg [2:0] alu_control
);

always @(*) begin

RegWR = 0;
MRD = 0;
MWR = 0;
MemtoReg = 0;
```

```

ALUSrc = 0;
Branch = 0;
Jump = 0;
RegDst = 1;
BranchType = 0;
alu_control = 3'b000;

case(opcode)

4'b0000: begin RegWR = 1; alu_control = 3'b000; end // AND
4'b0001: begin RegWR = 1; alu_control = 3'b001; end // OR
4'b0010: begin RegWR = 1; alu_control = 3'b010; end // ADD
4'b0011: begin RegWR = 1; alu_control = 3'b011; end // SUB

4'b0100: begin RegWR = 1; ALUSrc = 1; RegDst = 0; alu_control = 3'b000; end // ANDi
4'b0101: begin RegWR = 1; ALUSrc = 1; RegDst = 0; alu_control = 3'b001; end // ORi
4'b0110: begin RegWR = 1; ALUSrc = 1; RegDst = 0; alu_control = 3'b010; end // ADDi
4'b0111: begin RegWR = 1; ALUSrc = 1; RegDst = 0; alu_control = 3'b011; end // SUBi

4'b1000: begin RegWR = 1; alu_control = 3'b111; end // SLT
4'b1001: begin RegWR = 1; ALUSrc = 1; RegDst = 0; alu_control = 3'b111; end // SLTi

4'b1010: begin Branch = 1; BranchType = 0; alu_control = 3'b011; end // BEQ
4'b1011: begin Branch = 1; BranchType = 1; alu_control = 3'b011; end // BNE

4'b1100: Jump = 1; // J

4'b1110: begin RegWR = 1; MRD = 1; MemtoReg = 1; ALUSrc = 1; RegDst = 0; alu_control =
3'b010; end // LW
4'b1111: begin MWR = 1; ALUSrc = 1; alu_control = 3'b010; end // SW

default: begin RegWR = 0; MWR = 0; end // For Unused/Default

endcase

end

endmodule

```

7) Datapath (DP):

```

module DP(
input clk,
input clr,
input RegWR,
input MRD,
input MWR,
input MemtoReg,

```

```

input ALUSrc,
input Branch,
input Jump,
input RegDst,
input BranchType,
input [2:0] alu_control,
input [15:0] instruction,
output [3:0] pc_out,
output zero_flag
);

wire [1:0] rs = instruction[7:6];
wire [1:0] rt = instruction[5:4];
wire [3:0] imm = instruction[3:0];

wire [1:0] wreg = (RegDst) ? instruction[3:2] : instruction[5:4];

wire [3:0] r1, r2, aluB, alu_out, mem_out, wdata;

RF_4x4 rf(clk, clr, RegWR, rs, rt, wreg, wdata, r1, r2);

assign aluB = (ALUSrc) ? imm : r2;

ALU_4bit alu(r1, aluB, alu_control, alu_out, zero_flag);
DM_16x4 dm(clk, clr, MRD, MWR, alu_out, r2, mem_out);

assign wdata = (MemtoReg) ? mem_out : alu_out;

wire [3:0] pc_plus = pc_out + 1;
wire [3:0] branch_addr = pc_plus + imm;
wire [3:0] jump_addr = instruction[3:0];

wire take = (BranchType) ? ~zero_flag : zero_flag;

wire [3:0] pc_next = (Jump) ? jump_addr :
                    (Branch && take) ? branch_addr :
                    pc_plus;

PC_4bit pc(pc_out, pc_next, clk, clr);

endmodule

```

8) Microprocessor (MP):

```

module Processor(
input clk,
input clr,
input [15:0] instruction_manual,

```

```

input instr_sel
);

wire [3:0] pc;
wire [15:0] inst_mem;
wire [15:0] instruction;
wire RegWR, MRD, MWR, MemtoReg, ALUSrc, Branch, Jump, RegDst, BranchType;
wire [2:0] alu_control;
wire zero_flag;

IM_16x16 im(pc, inst_mem);
assign instruction = (instr_sel) ? instruction_manual : inst_mem;

CU cu(instruction[11:8], RegWR, MRD, MWR, MemtoReg, ALUSrc, Branch, Jump, RegDst,
BranchType, alu_control);

DP dp(clk, clr, RegWR, MRD, MWR, MemtoReg, ALUSrc, Branch, Jump, RegDst,
BranchType, alu_control, instruction, pc, zero_flag);

endmodule

```

9) Testbench for Microprocessor:

```

module testbench();
reg clk, clr, instr_sel;
reg [15:0] instruction_manual;

Processor uut(clk, clr, instruction_manual, instr_sel);

initial begin
clk = 0;
forever #5 clk = ~clk;
end

initial begin
clr = 1; instr_sel = 0; instruction_manual = 0;
#15 clr = 0;

#150;
end
endmodule

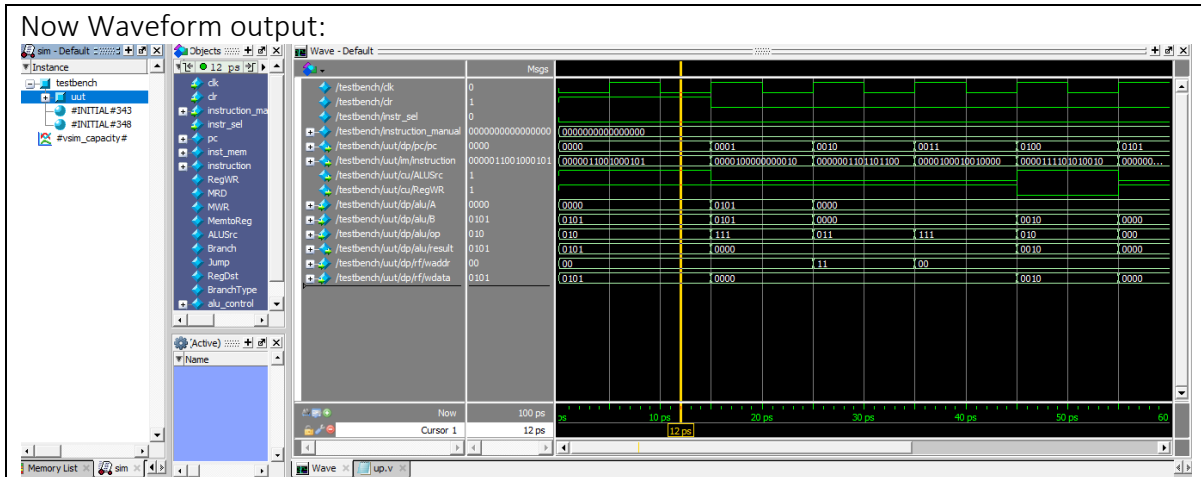
```

Verilog Output of the Waveforms of MP:

As an example, checking the ADDi functionality in Verilog. In which:

Function	Opcode	A1A0	Add/Sub	Cin	RegWR	MRD	MWR	RegDst	MemtoReg	ALUSrc	Branch
ADDi	0110	10	0	0	1	0	0	0	0	1	0

Now Waveform output:



As here we can see from the waveform, the ADDi functionality is working:

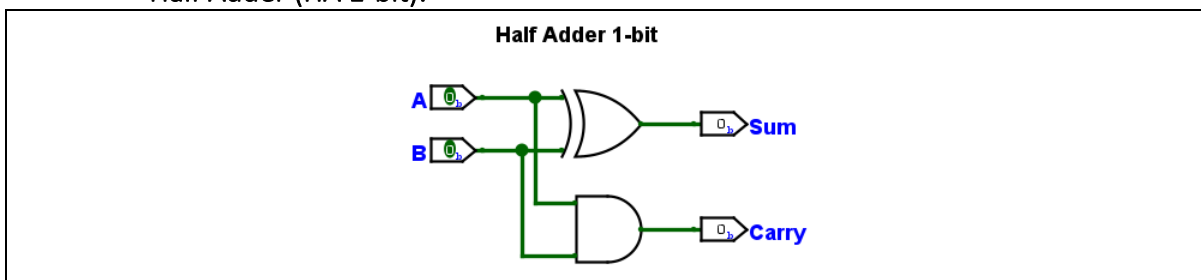
ALU A = 0000, B = 0101, Result = 0101

Other modules and parameters are also working correctly.

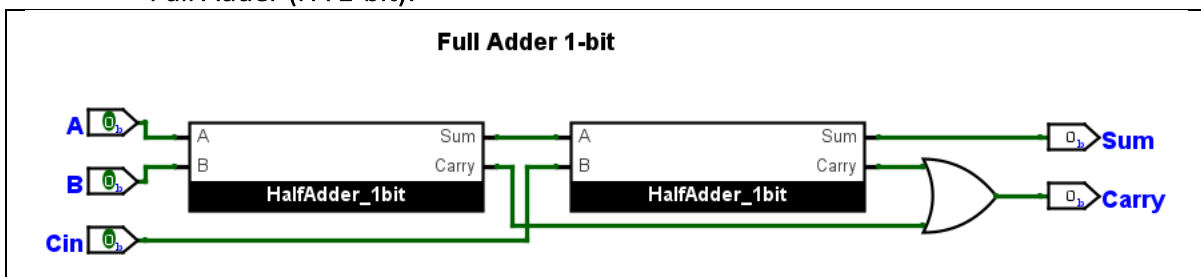
Logisim Design for each Module with its Sub-modules:

1) Arithmetic Logic Unit (ALU 4-bit):

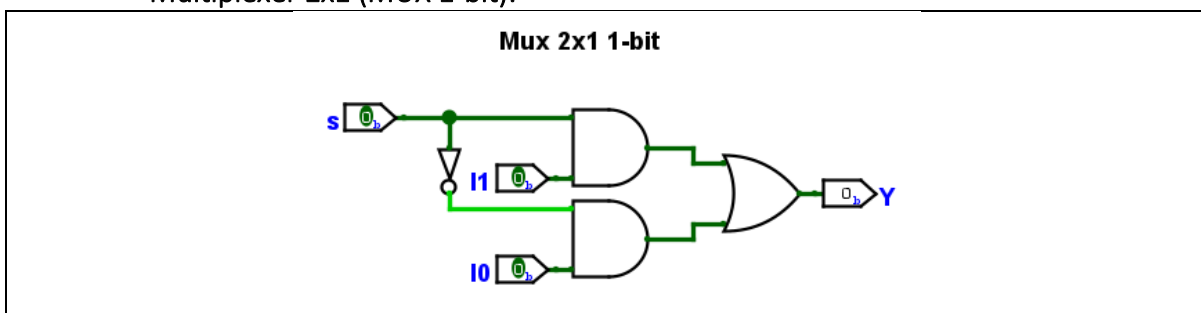
- Half Adder (HA 1-bit):



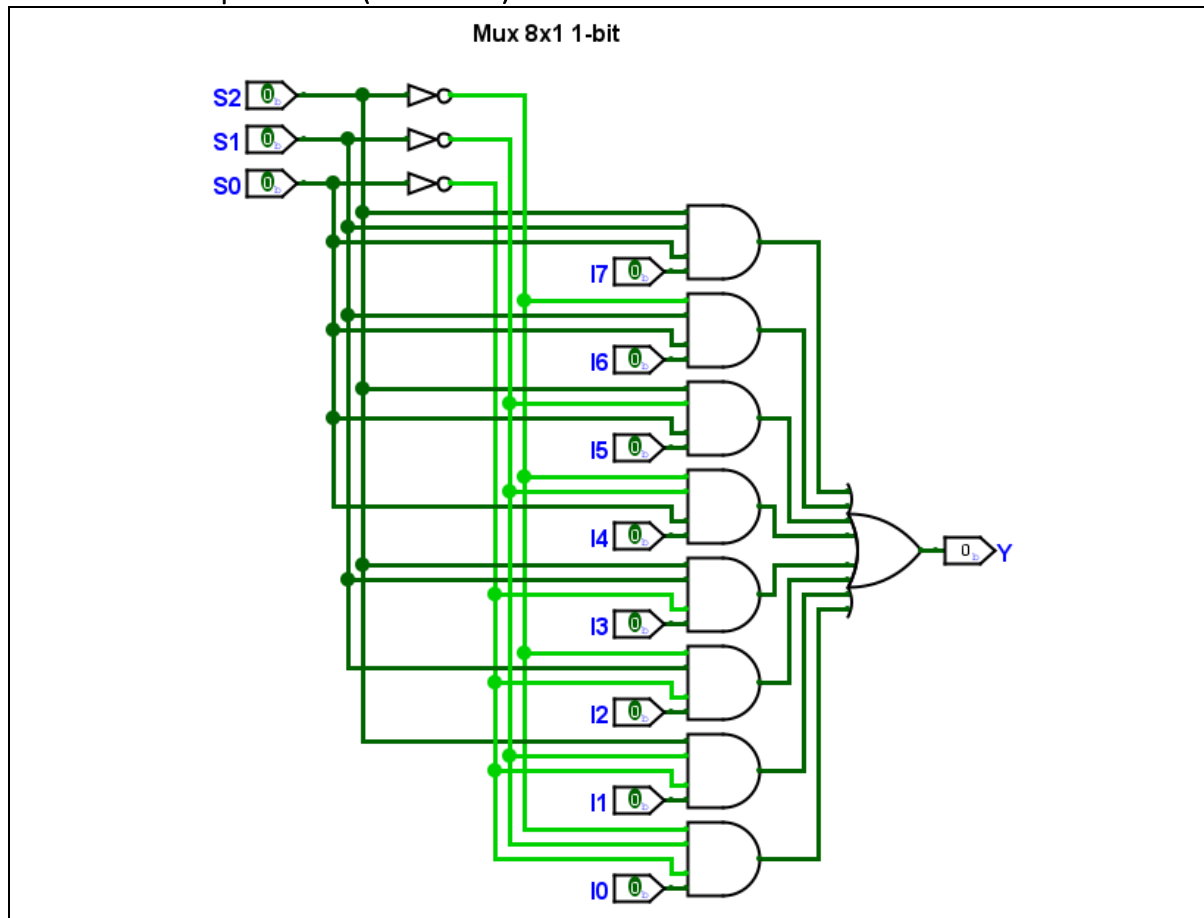
- Full Adder (FA 1-bit):



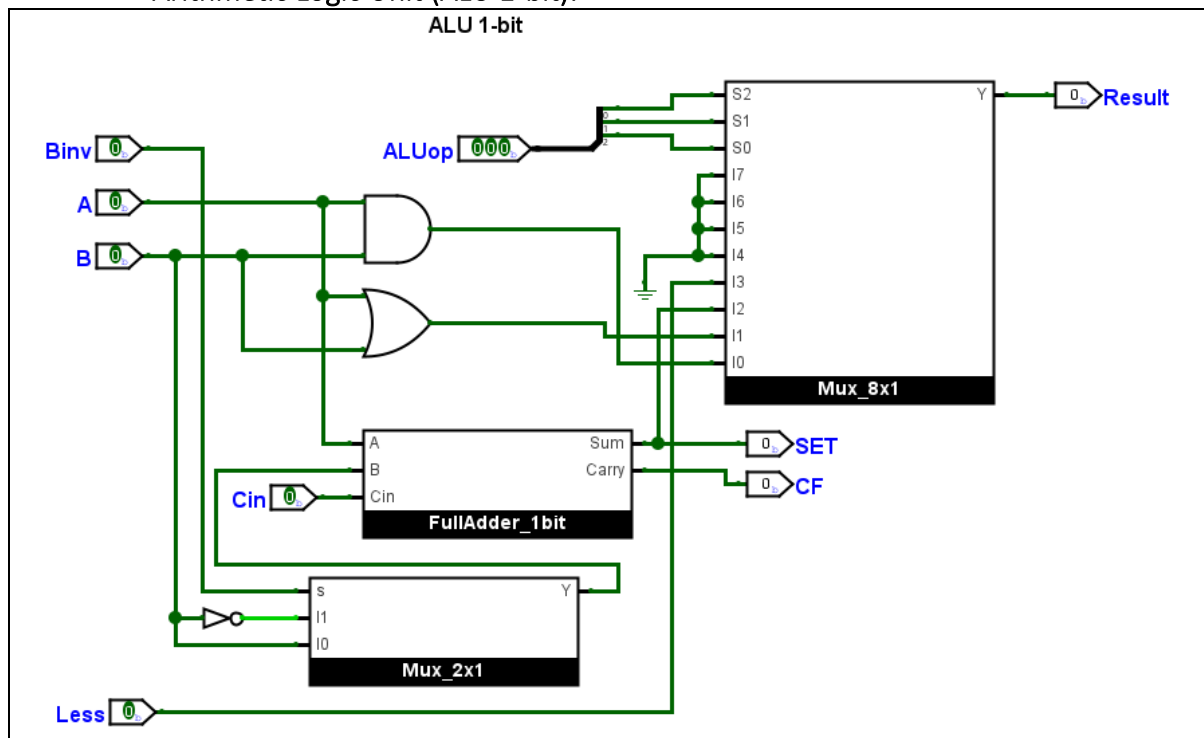
- Multiplexer 2x1 (MUX 1-bit):



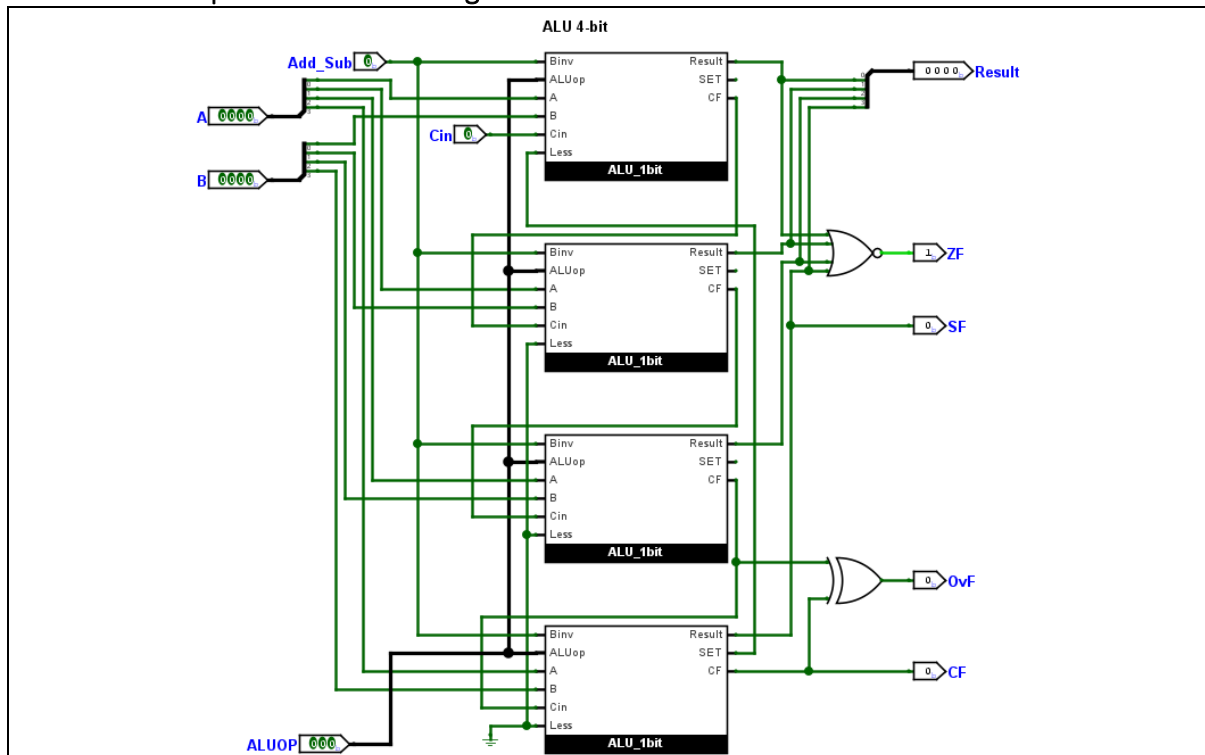
- Multiplexer 8x1 (MUX 1-bit):



- Arithmetic Logic Unit (ALU 1-bit):

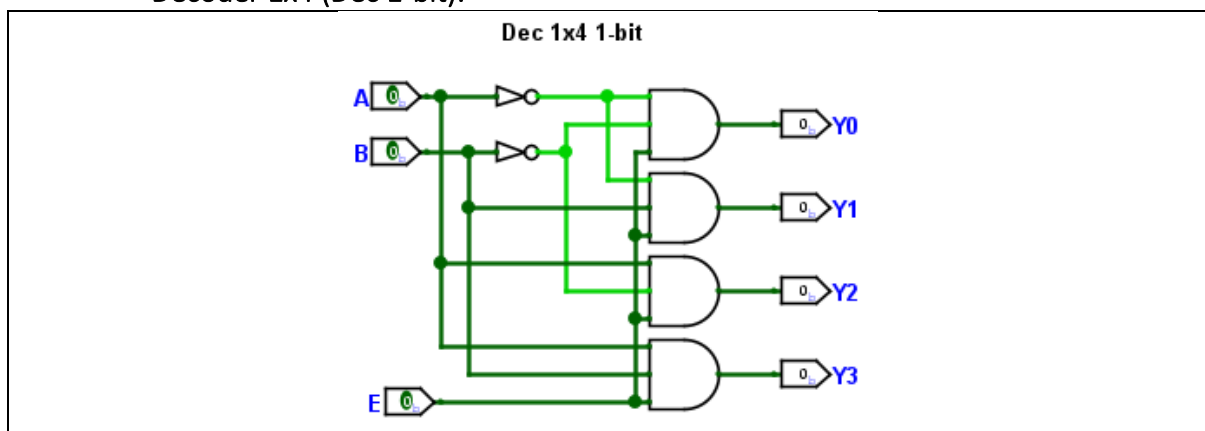


- Complete ALU 4-bit Design:

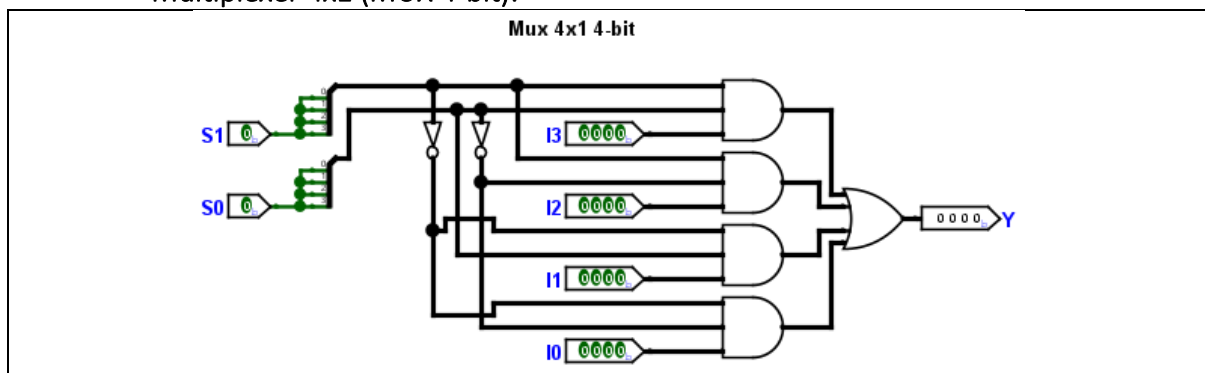


2) Register File (RF $2^4 \times 4$):

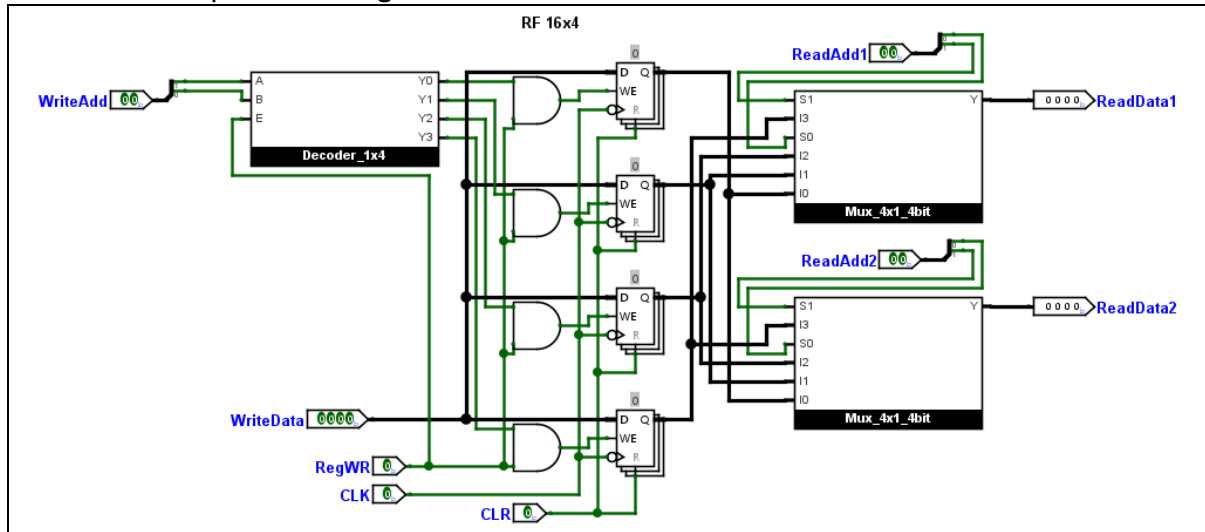
- Decoder 1x4 (Dec 1-bit):



- Multiplexer 4x1 (MUX 4-bit):

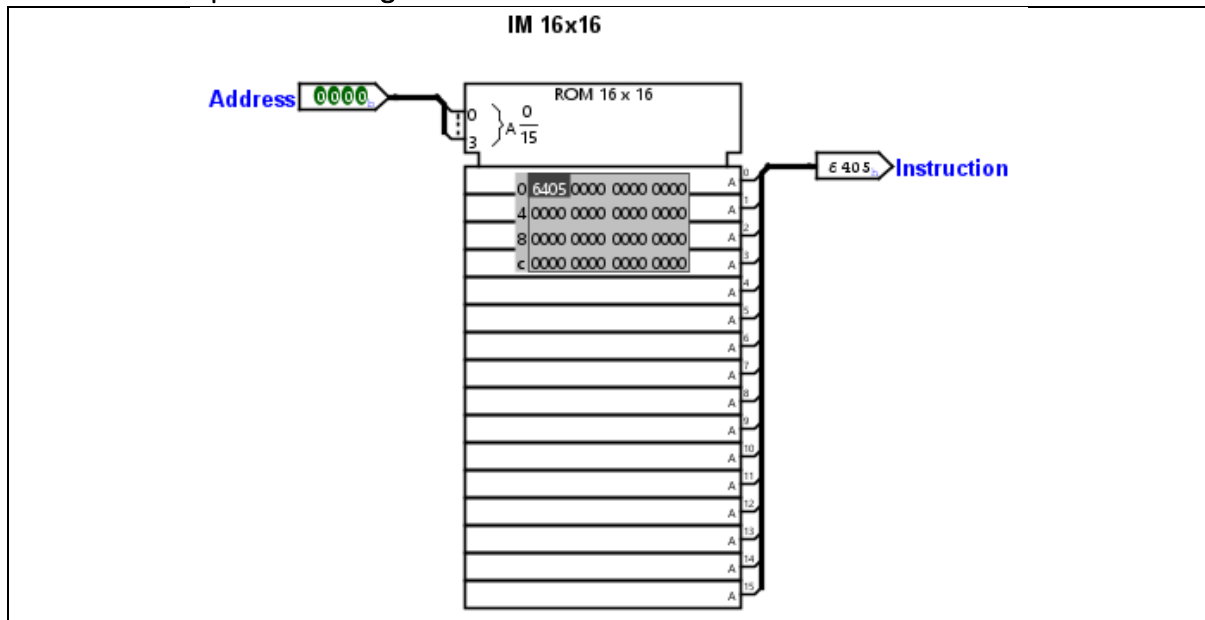


- Complete RF Design:



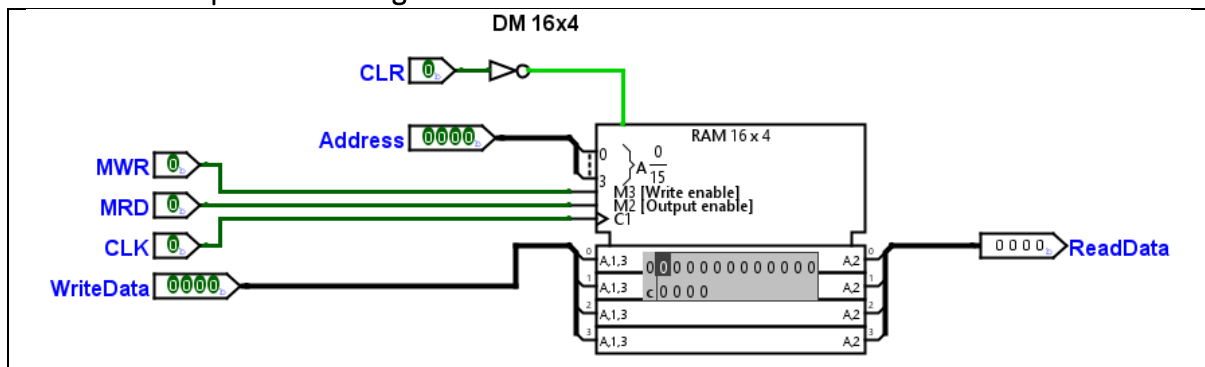
3) Instruction Memory (IM $2^4 \times 16$):

- Complete IM Design:



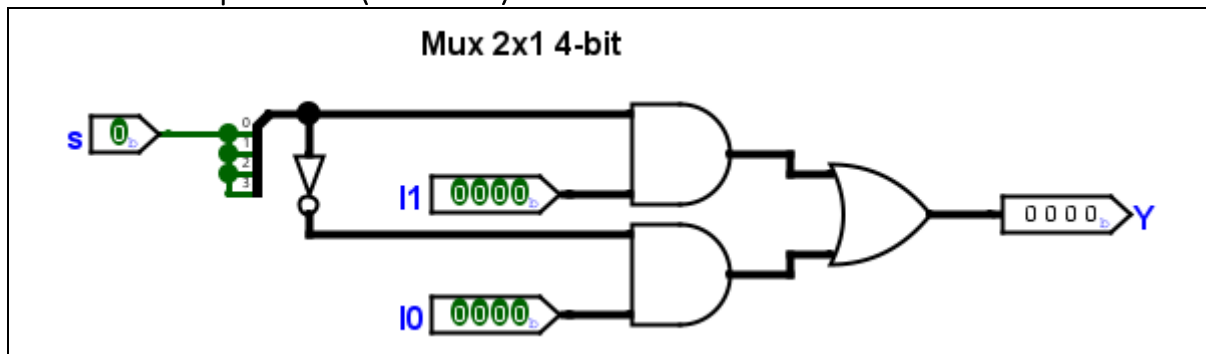
4) Data Memory (DM $2^4 \times 4$):

- Complete DM Design:

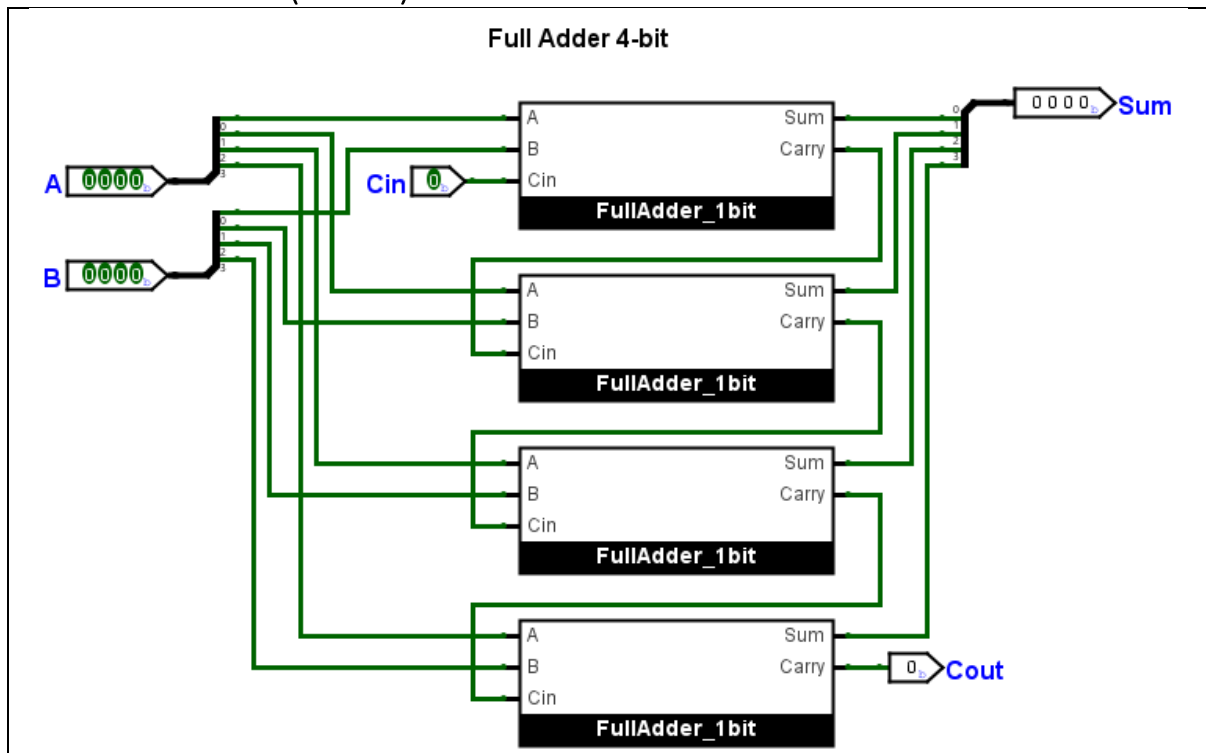


5) Program Counter (PC 4-bit):

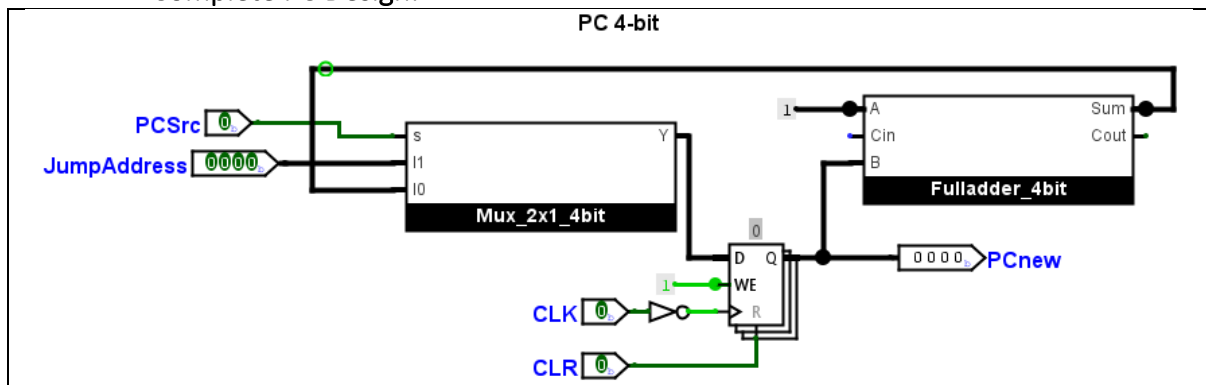
- Multiplexer 2x1 (MUX 4-bit):



- Full Adder (FA 4-bit):

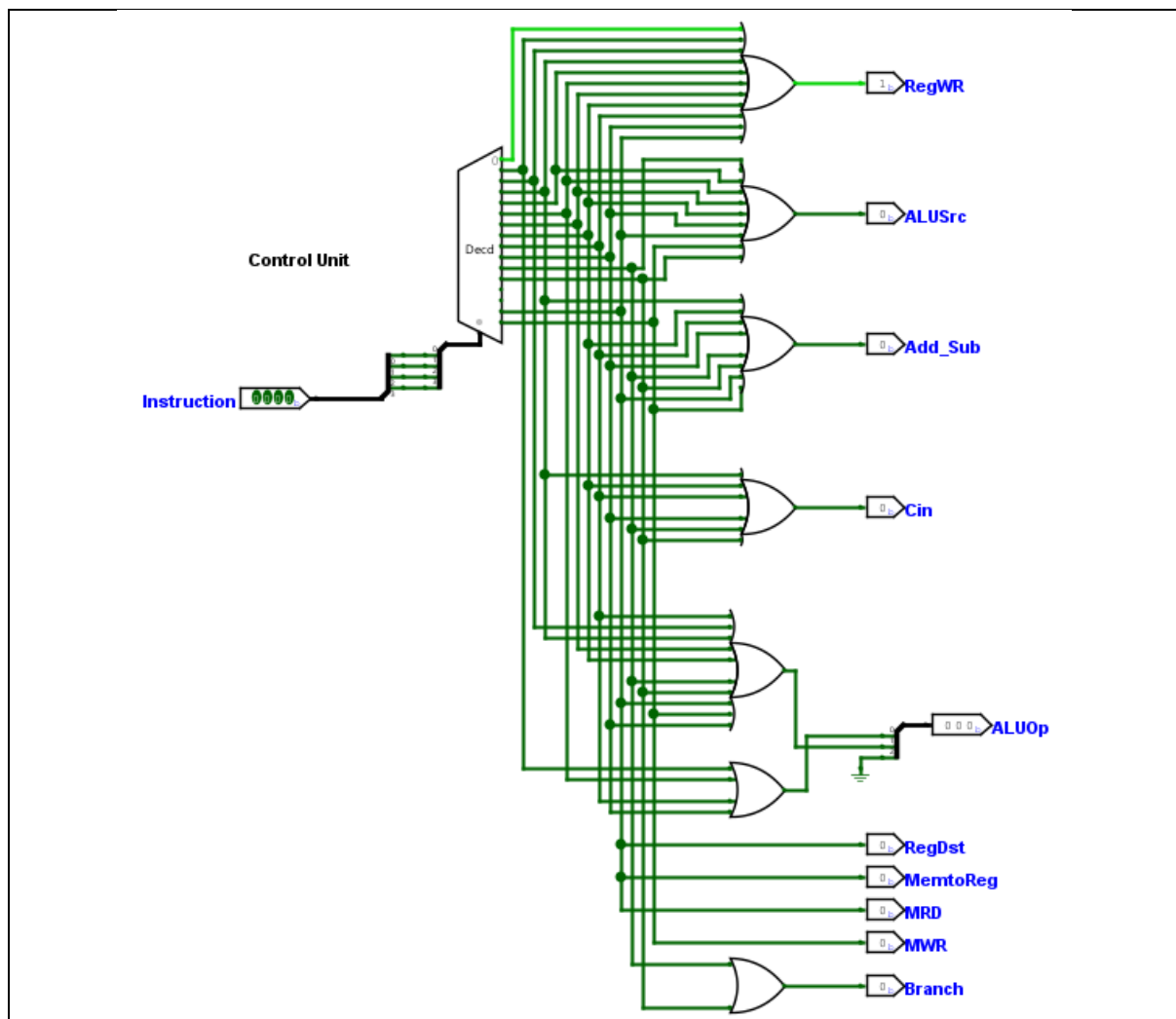


- Complete PC Design:



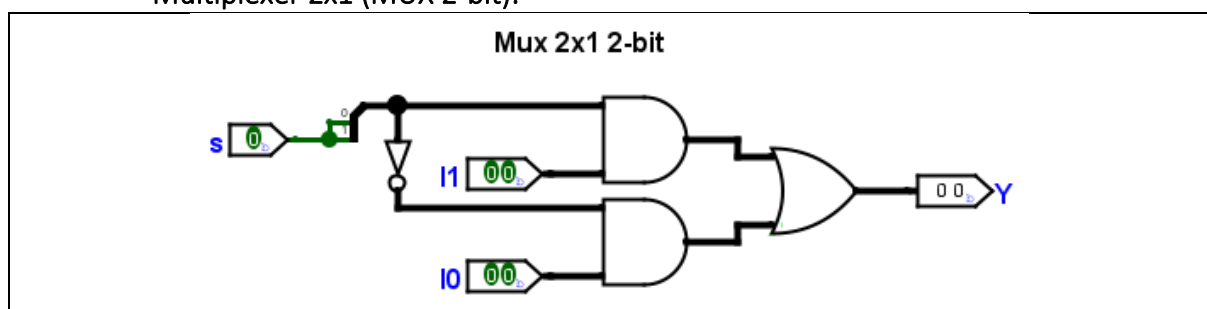
6) Control Unit (CU):

- Complete CU Design:

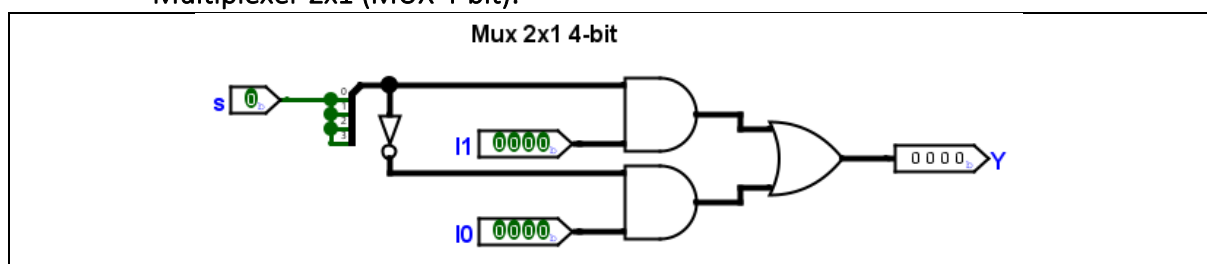


7) Microprocessor (MP):

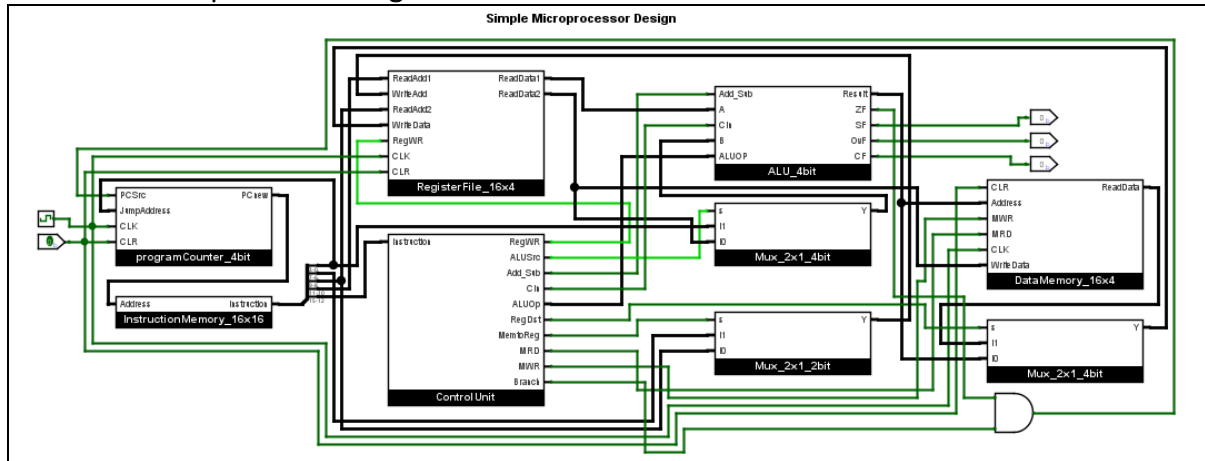
- Multiplexer 2x1 (MUX 2-bit):



- Multiplexer 2x1 (MUX 4-bit):



- Complete MP Design:

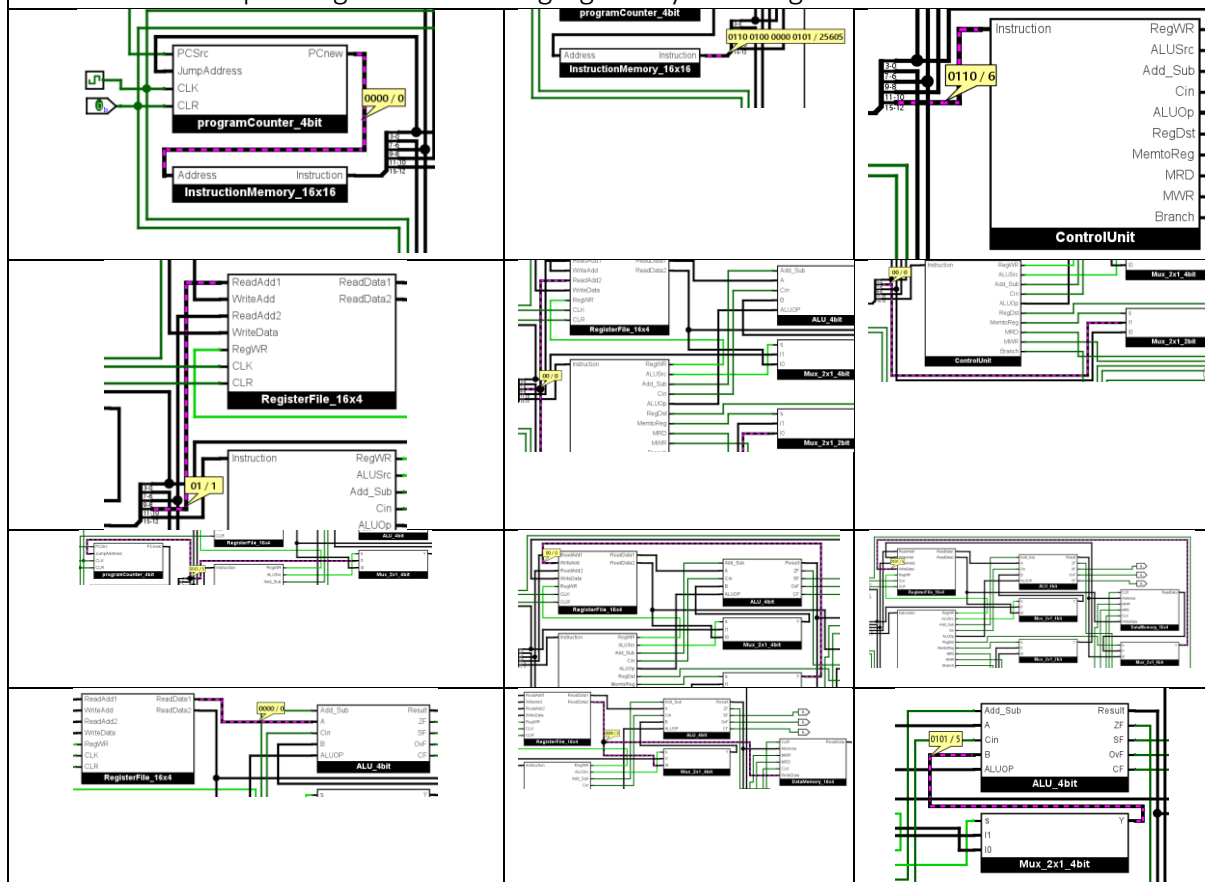


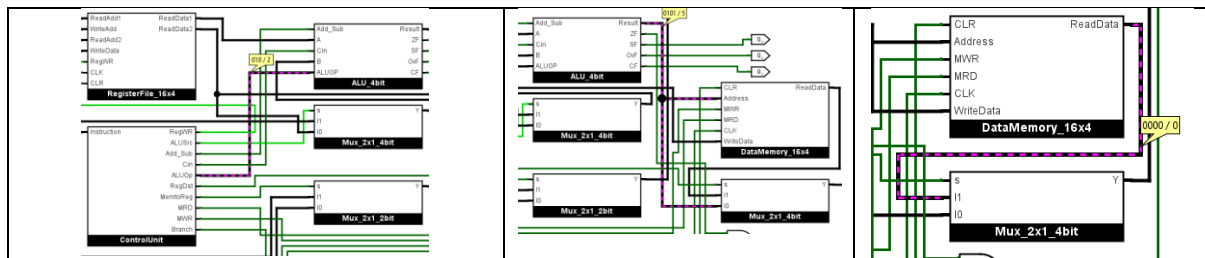
Logisim Output of Signals of MP Design:

As an example, checking the ADDi functionality in Logisim. In which:

Function	Opcode	A1A0	Add/Sub	Cin	RegWR	MRD	MWR	RegDst	MemtoReg	ALUSrc	Branch
ADDi	0110	10	0	0	1	0	0	0	0	1	0

Now with the help of finger tool checking signals by touching wires:





As here we can see from the signals, the ADDi functionality is working:

ALU A = 0000, B = 0101, Result = 0101

Other modules and parameters are also working correctly.

Complete Control Signals Table for 4-bit Microprocessor:

Function	Opcode	A1A0	Add/Sub	Cin	RegWR	MRD	MWR	RegDst (LW)	MemtoReg (SW)	ALUSrc (IMM)	Branch
AND	0000	00	X	X	1	0	0	1	0	0	0
OR	0001	01	X	X	1	0	0	1	0	0	0
ADD	0010	10	0	0	1	0	0	1	0	0	0
SUB	0011	10	1	1	1	0	0	1	0	0	0
ANDi	0100	00	X	X	1	0	0	0	0	1	0
ORi	0101	01	X	X	1	0	0	0	0	1	0
ADDi	0110	10	0	0	1	0	0	0	0	1	0
SUBi	0111	10	1	1	1	0	0	0	0	1	0
SLT	1000	11	1	1	1	0	0	1	0	0	0
SLTi	1001	11	1	1	1	0	0	0	0	1	0
BEQ	1010	10	1	1	0	0	0	X	X	0	1
BNE	1011	10	1	1	0	0	0	X	X	0	1
J	1100	XX	X	X	0	0	0	X	X	X	0
Unused	1101	XX	X	X	0	0	0	X	X	X	0
LW	1110	10	0	0	1	1	0	0	1	1	0
SW	1111	10	0	0	0	0	1	X	X	1	0